การเผยแพร่ข้อสอบ

☐ อนุญาต ☒ ไม่อนุญาต

เลขที่นั่งสอบ _____

วิชา

CPE334

Software Engineering

วันที่สอบ

27 ตุลาคม 2569

เวลาสอบ

13.00 - 16.00 (3 ชั่วโมง)

กลุ่มนักศึกษา

☒ ปกติ

☒ International

ชุดข้อสอบ (Optional) : 1

ชื่อ-นามสกุล : _____ **รหัสนักศึกษา :** _____

ภาควิชา/สาขาวิชา : _____ **กลุ่ม :** _____

คำชี้แจง

1. ข้อสอบมี **3 ส่วน 12 ข้อ คะแนนเต็ม 100 คะแนน** จำนวน 21 หน้า (รวมหน้าปก)
   - ส่วนที่ 1 Requirements และ Security ข้อ 1-4 รวม 25 คะแนน
   - ส่วนที่ 2 Project Planning และ Execution ข้อ 5-7 รวม 25 คะแนน
   - ส่วนที่ 3 UML/UI และ Git Workflow ข้อ 8-12 รวม 50 คะแนน
   - ท้ายเล่ม: แบบฝึกเสริม S1-S3 ไม่รวมใน 100 คะแนนและเวลา 3 ชั่วโมง

   ให้ทำคำตอบลงในข้อสอบ
2. เขียนชื่อและรหัสนักศึกษาลงในข้อสอบทุกหน้า
3. ใช้ปากกาหมึกสีน้ำเงินหรือสีดำ หากพื้นที่คำตอบไม่พอ ให้เขียนต่อด้านหลังของหน้าข้อสอบข้อเดียวกัน
4. ไม่มีการชี้แจงหรือแก้โจทย์ระหว่างสอบ หากไม่ชัดเจน ให้ตั้งสมมติฐานที่สมเหตุสมผลพร้อมอธิบาย

เงื่อนไขการสอบ

1. **ไม่อนุญาตให้นำเครื่องคำนวณเข้าห้องสอบ**
2. อนุญาตให้นำกระดาษ A4 1 แผ่นเข้าห้องสอบ โดยต้อง**เขียนชื่อและรหัสนักศึกษาบนกระดาษ** และ**ส่งพร้อมข้อสอบ** หากไม่ส่งกระดาษตามเงื่อนไข **จะหัก 5 คะแนนจากคะแนนสอบ**

* ข้อสอบได้ผ่านการพิจารณาจากคณะกรรมการประจำภาควิชาวิศวกรรมคอมพิวเตอร์แล้ว *

หัวหน้าภาควิชา	**อาจารย์ผู้สอน / ผู้ประสานงานรายวิชา**
.....	.....
	เบอร์โทรศัพท์ : .....

ส่วนที่ 1 Requirements และ Security (25 คะแนน)

ข้อ 1 วิเคราะห์ความต้องการของผู้มีส่วนได้ส่วนเสีย (8 คะแนน)

มหาวิทยาลัยต้องการพัฒนาระบบลงทะเบียน workshop ให้จำแนกข้อความ 1.1-1.4 เป็น functional, non-functional หรือ business/domain rule พร้อมเหตุผล จุดประสงค์ คุณภาพ วิธีตรวจและผลที่คาดหวัง แล้วปรับข้อความหรือให้เหตุผลว่าทำไมไม่ต้องปรับ เงื่อนไขหรือตัวเลขที่เสนอเพิ่มต้องระบุว่ารอยืนยันกับผู้เกี่ยวข้อง

1.1 (2 คะแนน) นักศึกษาที่เข้าสู่ระบบแล้วสามารถดู workshop ทั้งหมดที่เปิดให้ลงทะเบียนได้

1.2 (2 คะแนน) กระบวนการลงทะเบียนควรรวดเร็ว

ข้อ 1 (ต่อ) วิเคราะห์ข้อความที่เหลือตามหัวข้อเดิม

1.3 (2 คะแนน) ตามนโยบายมหาวิทยาลัย นักศึกษาลงทะเบียนได้ไม่เกิน 3 workshop ต่อภาคการศึกษา

1.4 (2 คะแนน) นักศึกษาสามารถยกเลิกการลงทะเบียน และผู้ประสานงานสามารถดูรายชื่อผู้เข้าร่วมได้

ข้อ 2 ประเมินข้อเสนอโครงการ WorkshopHub **(7 คะแนน)**

ปัจจุบันนักศึกษาสมัคร workshop ผ่านแบบฟอร์มออนไลน์แยกกัน ผู้ประสานงานรวบรวมข้อมูลด้วยมือ ทำให้ลงทะเบียนซ้ำและยืนยันล่าช้า WorkshopHub เป็นเว็บไซต์ให้นักศึกษาค้นหาและสมัคร workshop และให้ผู้ประสานงานจัดการรายชื่อ คาดว่าจะลดความผิดพลาดและงานจัดการ รุ่นแรกรองรับเฉพาะ workshop ฟรี ส่วน payment และใบรับรองพัฒนาภายหลัง ทีมวางแผนเสร็จก่อนภาคการศึกษาถัดไป

2.1 ประเมินองค์ประกอบทั้ง 8 ของข้อเสนอ **(4 คะแนน)**

ระบุองค์ประกอบทั้ง 8 ว่ามีแล้ว มีเพียงบางส่วน หรือยังไม่มี พร้อมหลักฐานหรือเหตุผลจากข้อความ

องค์ประกอบ	สถานะและหลักฐาน
Problem	
Users	
Solution	
Expected value	
Scope	
Risk	
Effort	
Evidence of completion	

2.2 เขียนข้อเสนอฉบับปรับปรุง **(3 คะแนน)**

รักษาปัญหาและขอบเขตรุ่นแรกเดิม เติมส่วนที่ยังขาด และแยกสิ่งที่เสนอเพิ่มออกจากข้อเท็จจริงที่ได้รับ

ข้อ 3 Role-Based Access Control **(4 คะแนน)**

หนึ่งบัญชีมีหนึ่ง role ไม่มี inheritance และปฏิเสธ operation ที่ไม่ได้รับสิทธิ์ Student ดู workshop ที่เผยแพร่และส่งคำขอลงทะเบียนตนเองได้ Coordinator ดู workshop ที่เผยแพร่ สร้างและแก้ไข workshop และส่งออกรายชื่อได้ ไม่มีข้อจำกัดตามเจ้าของ workshop เพิ่มเติม

3.1 เติม Allow หรือ Deny ให้ครบ **(2.5 คะแนน)**

Operation	Student	Coordinator
ดู workshop ที่เผยแพร่		
ส่งคำขอลงทะเบียนตนเอง		
สร้าง workshop		
แก้ไข workshop		
ส่งออกรายชื่อ		

3.2 การซ่อนปุ่มส่งออกจาก Student เพียงพอหรือไม่? **(1.5 คะแนน)**

อธิบายสิ่งที่ backend ต้องตรวจ และยกตัวอย่าง direct-request test พร้อมผลที่คาดหวังหนึ่งกรณี

ข้อ 4 วิเคราะห์ช่องโหว่ระบบส่งรายงาน **(6 คะแนน)**

ระบบส่งรายงานตรวจ PDF เฉพาะใน browser นักศึกษาข้ามการตรวจแล้วอัปโหลด script ปลอมเป็น PDF ระบบเก็บไฟล์ในโฟลเดอร์ที่รัน script ได้ เมื่อเปิด URL ของไฟล์ script จึงทำงานบน server และอ่านรายงานของคนอื่นได้

4.1 วิเคราะห์เหตุการณ์ **(3 คะแนน)**

ระบุผู้โจมตี assets, attack surface, trust boundary ที่ถูกข้าม และ security objective ที่เสียหาย

4.2 เสนอ control พร้อม negative test จำนวน 2 คู่ **(3 คะแนน)**

เสนอ control พร้อม negative test 2 คู่ แต่ละคู่ระบุวิธีป้องกัน วิธีทดสอบ และผลที่ควรได้ โดยเชื่อมกับช่องโหว่ในเหตุการณ์

ส่วนที่ 2 Project Planning และ Execution (25 คะแนน)

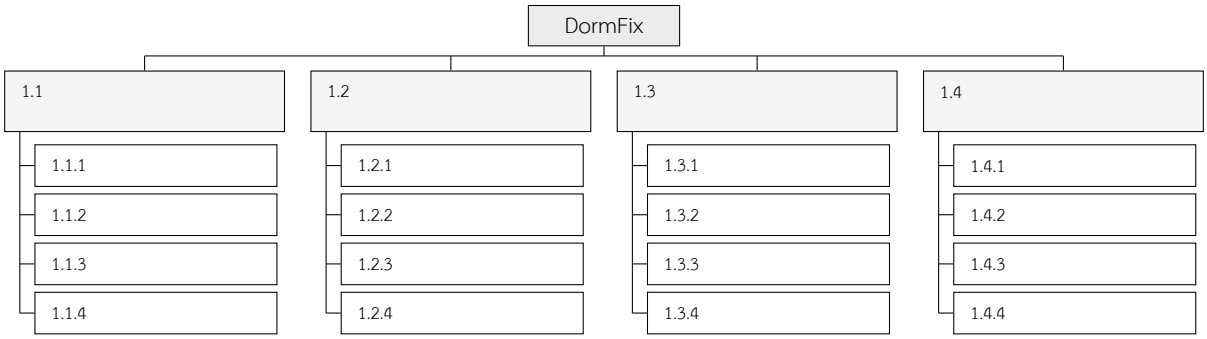
ข้อ 5 วางแผนโครงการ DormFix **(10 คะแนน)**

คุณเป็น PM ของระบบแจ้งซ่อมหอพักแบบ Waterfall ทีมประกอบด้วย Aom (BA), Bank (UX/UI), Chai (Developer), Dao (Developer/Tester) ผู้จัดการหอพักเป็น Sponsor และหัวหน้าช่างเป็นตัวแทน User

งาน	รายละเอียด	ผู้ทำ	วัน
A	เขียนแบบฟอร์มแจ้งซ่อมจากแบบที่อนุมัติ	Chai	4
B	ส่ง requirements ให้ผู้จัดการหอพักอนุมัติ	PM	1
C	ออกแบบหน้าจอจาก requirements ที่อนุมัติ	Bank	3
D	ทดสอบโปรแกรมสองส่วนร่วมกัน	Dao	2
E	เขียน requirements เป็นงานแรก	Aom	3
F	ส่งแบบทั้งสองให้หัวหน้าช่างอนุมัติ	PM	1
G	ออกแบบฐานข้อมูลจาก requirements ที่อนุมัติ	Chai	2
H	ส่งระบบที่ทดสอบแล้วให้ผู้จัดการหอพักตรวจรับ	PM	1
I	เขียน dashboard ช่างจากแบบที่อนุมัติ	Dao	3

5.1 จัด WBS ตาม milestone ที่บรรลุ **(3 คะแนน)**

ใส่ชื่อ milestone และตัวอักษรงาน A-I ตามลำดับ ใช้ทุกงานครั้งเดียว งานขนานเรียงลำดับใดก่อนก็ได้ เว้นช่องที่ไม่ใช้



5.2 เติม RACI ตามบันทึกประชุม **(2 คะแนน)**

ใช้ R, A, C, I หรือ “-” เมื่อไม่มีบทบาท ตามบันทึกประชุมด้านล่าง

- E: Aom ลงมือทำ PM รับผิดชอบผลสำเร็จ Bank ให้ความเห็นก่อนทำ Chai รับแจ้งหลังทำ Dao ไม่มีบทบาท
- C: Bank ลงมือทำ PM รับผิดชอบผลสำเร็จ Aom ให้ความเห็นก่อนทำ Chai รับแจ้งหลังทำ Dao ไม่มีบทบาท
- G: Chai ลงมือทำ PM รับผิดชอบผลสำเร็จ Bank ให้ความเห็นก่อนทำ Dao รับแจ้งหลังทำ Aom ไม่มีบทบาท
- D: Dao ลงมือทำ PM รับผิดชอบผลสำเร็จ Chai ให้ความเห็นก่อนทำ Aom รับแจ้งหลังทำ Bank ไม่มีบทบาท

Task	PM	Aom	Bank	Chai	Dao
E					
C					
G					
D					

5.3 วางกำหนดการเป็น Gantt chart **(5 คะแนน)**

ใช้ owner และจำนวนวันที่ให้ เริ่มจันทร์ที่ 4 มกราคม 2027 ทำงานจันทร์-ศุกร์ ไม่มีวันหยุดอื่น คุณและทีมทั้งสี่ทำ DormFix เต็มเวลา แต่ผู้เกี่ยวข้องภายนอกมีงานอื่น งานถัดไปเริ่มวันทำงานหลัง predecessors และ buffer จบ ใส่ task buffer 1 วันและ project buffer 2 วันเฉพาะจุดจำเป็น Milestone อยู่วันสุดท้ายที่ทำเครื่องหมายของงานสุดท้ายในกลุ่ม; milestone สุดท้ายอยู่วันสุดท้ายของ project buffer โดยไม่เพิ่มวันทำงาน กรอก No. ตาม WBS, Task, Owner, Start และ Due รวม buffer ใช้ X แทนวันทำงาน, - แทน buffer, * แทน milestone แสดง dependencies โดยใช้จุดเชื่อมข้ามตารางได้ เว้นแถวที่ไม่ใช้ และอธิบายด้านล่าง

ข้อ 5.3 ตารางคำตอบ Gantt chart

January 2027: Weeks 1-2

No.	Task	Owner	Start	Due	4	5	6	7	8	11	12	13	14	15

January 2027: Weeks 3-4

No.	Task	Owner	Start	Due	18	19	20	21	22	25	26	27	28	29

คำอธิบาย (1 คะแนน รวมในข้อ 5.3): เพราะเหตุใดจึงวาง buffer เช่นนี้ และ milestone ไม่ใช้เวลาทำงาน?

ข้อ 6 Backlog grooming: จองสนามแบดมินตัน (10 คะแนน)

Issue #12 “Book a badminton court”: Client ต้องการให้นักศึกษาจองสนามครั้งละ 1 ชั่วโมงโดยไม่ต้องต่อคิว ระบบมีข้อมูลสนามและเวลาเปิดให้บริการแล้ว นักศึกษาแต่ละคนถือการจองได้ไม่เกิน 1 รายการต่อวัน ตัวอย่าง: สนาม 2 ว่างเวลา 17:00 วันนี้และต้องแสดง Booked เมื่อจองสำเร็จ ทีมยังไม่กำหนดผู้รับผิดชอบ milestone หรือ estimate ให้เลือก label ที่เหมาะสม ใส่ “-” ในช่องที่ไม่มีข้อมูล และแยกคำถามติดตามออกจากข้อเท็จจริง

6.1 เขียน user story: role, function และ value (2 คะแนน)

6.2 ระบุ assumptions/details จากข้อมูล client (2 คะแนน)

6.3 เขียน Acceptance Criteria แบบ Gherkin 2 สถานการณ์ (4 คะแนน)

เขียน Gherkin 2 สถานการณ์: จองสำเร็จตามตัวอย่าง และขอจองเพิ่มเมื่อมีการจองในวันนั้นแล้ว

6.4 กำหนด metadata ของ issue (2 คะแนน)

Assignees	Labels	Milestone	Estimate

ข้อ 7 Sprint planning: ShuttleNow **(5 คะแนน)**

คุณเป็น Scrum Master เป้าหมาย Sprint 6: “นักศึกษาติดตามรถรับส่งและทราบเวลาที่รถจะมาถึงได้” Capacity เท่า Sprint ก่อน คือ 16 points Backlog เรียงตามความสำคัญ Anchor “ดูรายการเส้นทางรถรับส่ง” = 3 points บน scale 1, 2, 3, 5, 8, 13, 21 ใหญ่ขึ้น/เล็กลงหมายถึงขั้นบน scale

7.1 ประเมิน story points และเลือกงานเข้า Sprint 6 **(3 คะแนน)**

เติม points และ Yes/No ให้ครบ

ลำดับ	User story	เทียบ anchor	Points	เลือก
1	ดูตำแหน่งรถสด	ใหญ่ขึ้น 2 ขั้น		
2	จ่ายค่าโดยสาร QR	เท่ากัน		
3	เวลาถึงป้ายฉัน	ใหญ่ขึ้น 1 ขั้น		
4	เตือนก่อนรถถึง	เท่ากัน		
5	เวลาถึงทุกป้าย	ใหญ่ขึ้น 1 ขั้น		
6	แก้รายการถูกตัดบนมือถือ	เล็กลง 1 ขั้น		

7.2 รวม points ของงานที่เลือก **(1 คะแนน)**

_____

7.3 เหตุผลที่เลือกและไม่เลือกแต่ละงาน **(1 คะแนน)**

ส่วนที่ 3 UML/UI และ Git Workflow (50 คะแนน)

ใช้ข้อกำหนดต่อไปนี้สำหรับข้อ 8-10

ข้อกำหนด: ระบบลงทะเบียนเรียน KU

R1. ผู้ใช้ทุกคนต้อง log in ด้วย KU account ผ่าน KU Single Sign-On (KUSSO) ซึ่งให้ข้อมูล role เช่น Student, University Admin และ Instructor

R2. Department Admin สร้าง course ใหม่ได้ โดยระบุ course code, course name, description, credits และ fee

R3. Department Admin กำหนด course offerings โดยเลือก semester และ course code แล้วกรอก sections list: section number, room, class time, display notes และ maximum students รายวิชาที่เปิด 4 sections จะสร้าง 4 course offerings

R4. University Admin สร้าง semester และกำหนด currently active semester ได้

R5. Student เปิด Registration Form ของ active semester โดยเริ่มต้นยังไม่มี offering ระบบใช้ logged-in username ดึง student ID, name, degree program, email และ login username จาก KU Student Info System

R6. Student เพิ่ม offerings หลายรายการผ่าน Course Offering List View ได้ total registration fee ปรับตามรายวิชาที่เพิ่ม

R7. Course Offering List View แสดง course ID, course code, course name, credits, fee, semester, section, room, class time, display notes, maximum students, number registered และ seats remaining นักศึกษากรองและเลือกเพิ่มลงในฟอร์มได้

R8. ฟอร์มเริ่มในสถานะ Draft นักศึกษาบันทึกและกลับมาแก้ไขได้ก่อน final submission

R9. เมื่อ submit ระบบตรวจที่นั่งทุก offering หาก section ใดเต็ม submission ล้มเหลวและแจ้งเตือนให้แก้ไขแล้ว submit ใหม่ หากทุกรายการมีที่นั่งจึงไปชำระเงิน

R10. ชำระด้วย credit card ผ่าน KBank Payment Gateway หรือ bank transfer ผ่าน Bangkok Bank Payment Gateway (BBLPaymentGateway)

R11. หลังชำระสำเร็จ ฟอร์มเป็น Done และแก้ไขออนไลน์ไม่ได้ นักศึกษาได้รับ PDF ของ registration ทาง email ผ่าน KU Email (Google Workspace)

ข้อ 8 Sequence Diagram **(10 คะแนน)**

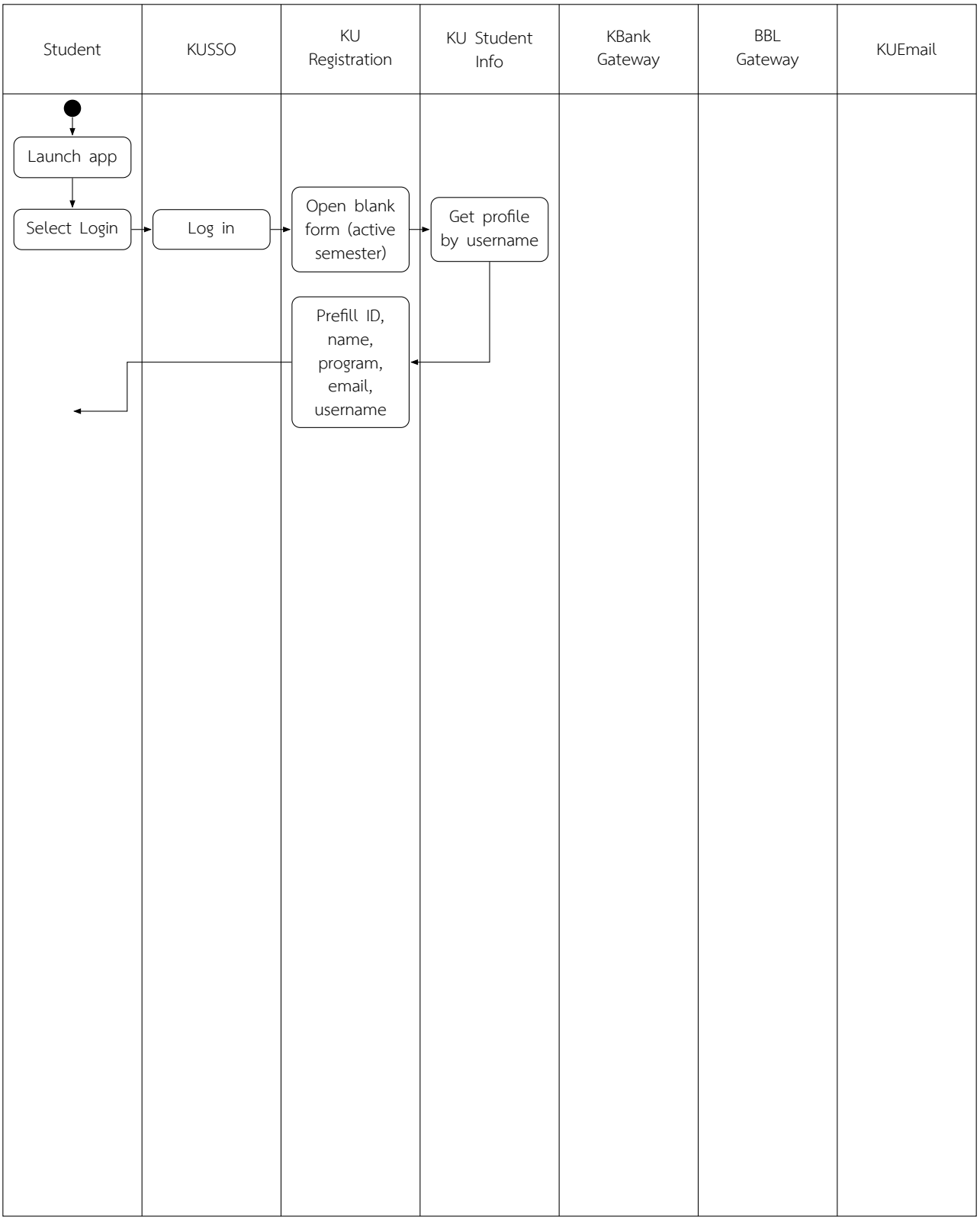
เติมแผนภาพตามเหตุการณ์ด้านล่าง แสดง messages พร้อมทิศทางและผลการทำงานของระบบให้ครบ

1. Student เข้าสู่ระบบ
2. ระบบพบ Draft ของภาคปัจจุบันที่มี 2 offerings และเปิดให้ทำต่อ
3. Student เพิ่มอีก 1 offering
4. Student submit แต่หนึ่งใน 3 offerings เต็ม จึงล้มเหลว
5. ระบบแจ้ง Student
6. Student เปลี่ยน offering ที่เต็มเป็น section อื่นแล้ว submit ใหม่
7. ครั้งนี้ submit สำเร็จ
8. Student ชำระด้วย bank transfer
9. การลงทะเบียนเสร็จสมบูรณ์



ข้อ 9 Activity Diagram **(10 คะแนน)**

แสดง workflow ใน session เดียว: Student เข้าระบบครั้งแรกของภาค ได้ฟอร์มว่างพร้อมข้อมูลตนเอง เลือกวิชา ชำระสำเร็จ และได้รับยืนยัน ให้แสดง decision branches เช่นวิธีชำระ พร้อม guards และ loops เช่นแก้ไขแล้ว resubmit จนสำเร็จ ใช้ decision nodes แบบ flowchart และวางกิจกรรมใน lane ผู้รับผิดชอบ



ข้อ 10 ออกแบบ List View และ Form View **(10 คะแนน)**

ออกแบบ generic UI สำหรับ Student ตาม R5-R11 ใช้ ***** นำหน้า required fields ใช้ textboxes สำหรับ editable fields และขีดเส้นเฉียงภายในช่อง non-editable หรือ calculated

ตัวอย่างรูปแบบ UI (Invoice): ใช้ดูสัญลักษณ์ แล้วออกแบบ fields และ controls ของ Registration ตามข้อกำหนด

[Save] [Clear] [Close]

Invoice No. *Invoice Date: [Date]

*Customer Code: [LoV] Customer Name:

Notes:

	*Product Code	Name	*Quantity	Unit Price	Price	
1	[LoV]					+ x

Total Due:

10.1 Course Offering List View **(4 คะแนน)**

แสดงข้อมูลตาม requirements การกรอง และเลือก offering กลับสู่ฟอร์ม ใช้ตารางหรือการ์ดพร้อมส่วนรายละเอียดได้

ข้อ 10 (ต่อ): ออกแบบแบบฟอร์มลงทะเบียน

10.2 Student Registration Form View **(6 คะแนน)**

Semester เป็นภาคปัจจุบัน ข้อมูล Student มาจากผู้ใช้ที่เข้าสู่ระบบ แสดง header, line items, fee, controls และการแสดงผลเมื่อ Draft, รายการเต็ม และ Done

ข้อมูลร่วมสำหรับข้อ 11-12

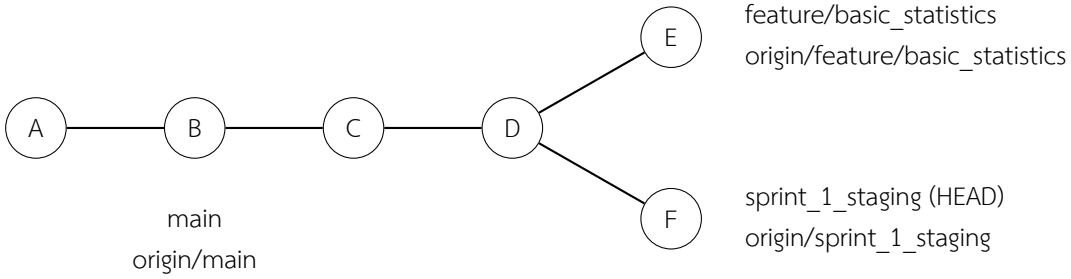
ExamScore เป็น Python CLI อ่านรายชื่อและคะแนน 0-100 จาก CSV คำนวณสถิติและอัตราผ่าน ใช้ main สำหรับ release, sprint staging รวมงาน และ feature branches แยกงาน

สัญลักษณ์และข้อสมมติ

HEAD ระบุ local branch ที่ checkout; origin/* บันทึกสถานะ remote ล่าสุดที่เครื่องทราบ switch เปลี่ยน HEAD โดยไม่ย้าย branch pointers สมมติ working directory สะอาด คำสั่งสำเร็จ ไม่มี conflict คำสั่งไม่ติดต่อ GitHub เว้นแต่ระบุ fetch, push หรือ remote command แต่ละข้อเริ่มจาก graph ของตน **Stage ด้วยชื่อไฟล์ ห้ามใช้ `git add .`**

ข้อ 11 ทำนาย repository graph (10 คะแนน)

จากข้อมูลร่วมข้างต้น feature และ sprint แตกแขนงจาก D ให้ merge สำเร็จและสร้าง G ตามคำสั่งด้านล่าง



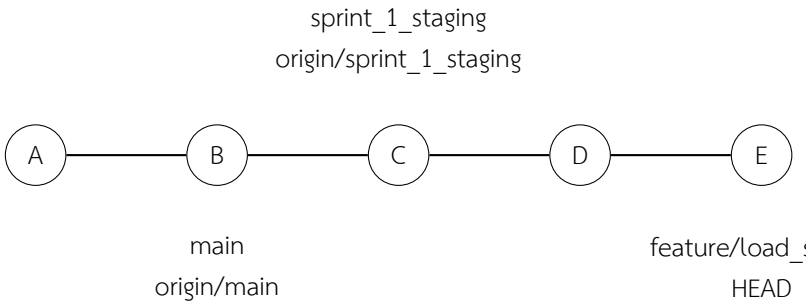

```
git merge feature/basic_statistics
git switch main
```

วาด graph สุดท้ายให้มี parents ทั้งสองของ G, local branches, origin/* และ HEAD ระบุประเภท merge พร้อมเหตุผล และผลของ switch ต่อ HEAD และ branch pointers

ประเภท merge เหตุผล ผลของ switch:

ข้อ 12 เขียนคำสั่งและวาด graph **(10 คะแนน)**

ทำ 12.1-12.3 ต่อเนื่องกัน แต่ละขั้นเริ่มจากผลของขั้นก่อน



12.1 เปลี่ยนไป sprint_1_staging แล้ววาด graph **(2 คะแนน)**

คำสั่ง:

12.2 รวม feature/load_scores เข้า branch ปัจจุบัน **(3 คะแนน)**

รวม feature/load_scores เข้า branch ปัจจุบัน เขียนคำสั่ง ระบุประเภท merge และวาด graph หลัง merge

คำสั่งและประเภท merge:

12.3 ตรวจการเปลี่ยนแปลง stage และ commit **(5 คะแนน)**

จาก 12.2 ผู้พัฒนาแก้ `score_data.py` และ `scores.csv` แล้วสร้าง `validation_notes.txt` เขียนคำสั่งตรวจการเปลี่ยนแปลง stage เฉพาะสามไฟล์ และสร้าง F ด้วยข้อความ `Document score validation cases` แล้ววาด pointers และ HEAD ให้ครบ

คำสั่ง:

Graph หลังสร้าง F:

แบบฝึกเสริมนอกคะแนน

S1-S3 ใช้ฝึกประเภทที่ไม่ได้เลือกในชุดหลัก ไม่รวมใน 100 คะแนนหรือเวลา 3 ชั่วโมง ข้อ 1-12 ยังคงเป็นชุดจำลองสอบเดิม

S1 Use Case Diagram: University Grades Submission

ข้อมูลสำหรับทำโจทย์ (เรียบเรียงจาก Lecture 4 หน้า 64-65)

1. Instructor เข้าระบบด้วย KU account ผ่าน KU Single Sign-On เฉพาะบัญชีที่ถูกต้องเท่านั้นที่เข้าใช้งานได้
2. Instructor เลือก semester แล้วดู courses และ sections ที่เปิดใน department ของตนจาก KU Registration System
3. เมื่อเลือก course และ section(s) ระบบดึงรายชื่อนักศึกษาที่ลงทะเบียนจาก KU Registration System มาใส่ grade sheet ได้
4. Grade sheet มี header (Course, Semester) และรายชื่อนักศึกษา เริ่มใน Draft Instructor กรอกหรือแก้เกรด และเลือกพิมพ์สำเนาเก็บไว้ได้
5. เมื่อกรอกเกรดครบ Instructor เปลี่ยน sheet เป็น Ready แล้วจึง submit ได้; ห้าม submit จาก Draft
6. เมื่อ submit ระบบอัปเดต transcript ของนักศึกษาทุกคน แจ้งเกรดแต่ละคนทาง email ยืนยันผลสำเร็จ และเปลี่ยน sheet เป็น Submitted

ระบบภายนอกทั้งห้า

KU Single Sign-On	ตรวจบัญชีและจัดการรหัสผ่าน
KU Registration System	ให้ข้อมูล courses/sections และผู้ลงทะเบียน
KU Transcript System	บันทึกเกรดอย่างเป็นทางการ
KU Student Information System	ให้ข้อมูลติดต่อ เช่น email ของนักศึกษา
KU Email Service (Google Workspace)	ส่ง email แจ้งเกรด

คำสั่ง: วาด Use Case Diagram ในหน้าถัดไปให้ครอบคลุมเป้าหมายของ Instructor และระบบภายนอก แสดง system boundary, actors และ associations ให้ถูก หากข้อกำหนดรองรับ ให้แสดง **<<include>>**, **<<extend>>** หรือ generalization ด้วยทิศทางและสัญลักษณ์ที่ถูกต้อง; ระบุเงื่อนไขของ **<<extend>>** และอย่าสร้างความสัมพันธ์ที่โจทย์ไม่ได้กำหนด

อธิบายสั้น ๆ: อธิบายความสัมพันธ์ที่ใช้ว่าเป็นขั้นบังคับหรือทางเลือก เงื่อนไขใดทำให้ submit ได้ และมี generalization ที่ข้อกำหนดรองรับหรือไม่

S1 (ต่อ): พื้นที่วาด Use Case Diagram

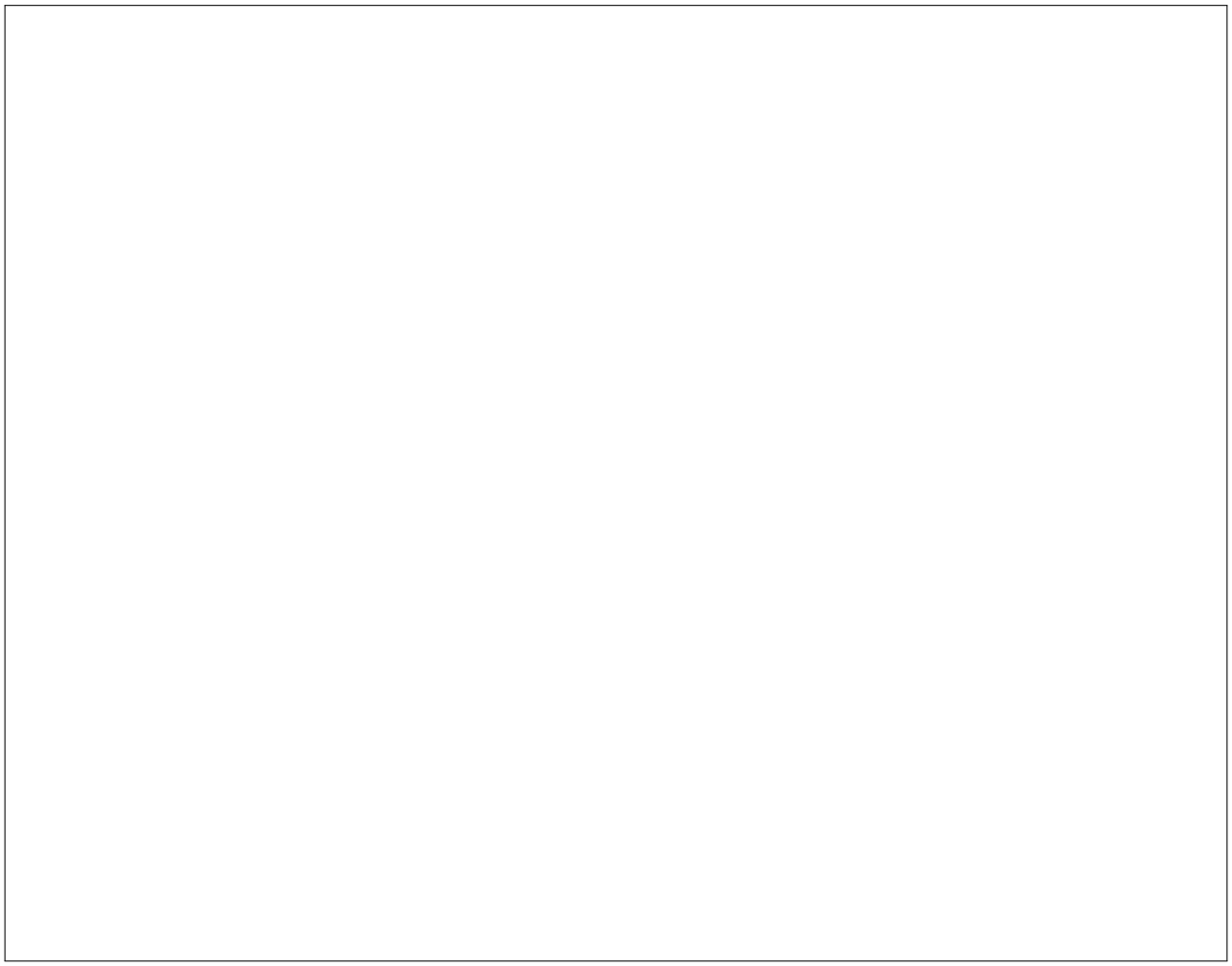
University Grades Submission แบบฝึกเสริม ไม่รวม 100 คะแนน

S2 State Diagram: Service Ticket

แบบฝึกเสริม ไม่รวม 100 คะแนน ใช้เส้นทางทำงานของตัวอย่าง Lecture 4 หน้า 61-62 และข้อมูลจากหน้า 45-46

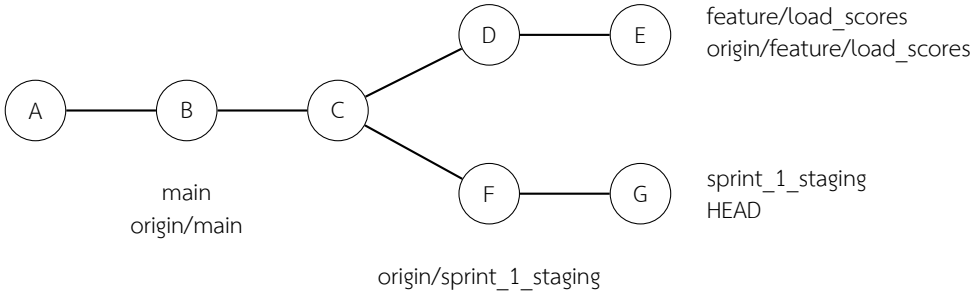
1. เมื่อบันทึก Ticket ใหม่ System สร้าง Action Record แรกโดยอัตโนมัติ กำหนด status เป็น Requested และ New Due Date เท่ากับ Initial Due Date
2. CCE ทำ action ได้เฉพาะ Requested: อนุมัติเป็น Approved หรือปิดเป็น Closed by CCE พร้อม notes/วันนัดใหม่ได้ตามกรณี
3. ตามเส้นทางตัวอย่าง Programmer เริ่มงานจาก Approved ให้เป็น In Progress แล้วเพิ่ม progress actions เพื่อบันทึก notes หรือวันนัดใหม่ได้โดยยังเป็น In Progress
4. เมื่อทำงานเสร็จ Programmer ปิดเป็น Closed by Programmer และกำหนด New Due Date เป็น completion date
5. หลังบันทึกทุก action ระบบอัปเดต Last Status และ Last Due Date ของ Ticket จาก action ล่าสุด; action ที่บันทึกแล้วแก้ไขหรือลบไม่ได้
6. Ticket ที่ Closed by CCE หรือ Closed by Programmer เพิ่ม action ไม่ได้อีก Customer และ IT Admin ไม่ได้ทำ actions สำหรับเปลี่ยนสถานะเหล่านี้

คำสั่ง: วาดเฉพาะวงจรสถานะของ Ticket หนึ่งใบ แสดง initial/final nodes, states, transitions และ self-loop ระบุชื่อ event และ actor ในวงเล็บเหลี่ยมตามแนว เช่น `approve [CCE]` ไม่ใช้ Customer/CCE/Programmer เป็นชื่อ state และไม่เติม state ที่โจทย์ไม่กำหนด



S3 Git: อธิบาย repository graph

แบบฝึกเสริม ไม่รวม 100 คะแนน เรียบเรียงจาก Samples ข้อ 2.2 หน้า 8-9 ใช้ ExamScore และข้อสมมติ Git เดียวกับข้อ 11-12 ทุกคำถามอ่านจาก snapshot นี้แยกกัน เว้นแต่ระบุให้รันคำสั่ง



S3.1 local branch ใดถูก checkout และ HEAD ระบุ commit ใด?

S3.2 commit ใดบนปลายสาย local sprint ยังไม่อยู่ในประวัติที่ origin/sprint_1_staging อ้างถึง? อธิบายจากตัวชี้

S3.3 ไฟล์ใน working directory ตอนนี้มาจาก commit ใด?

S3.4 ถ้ารัน `git merge feature/load_scores` ตอนนี้ fast-forward ได้หรือไม่? ต้องรวมแบบใดและเพราะอะไร?

S3.5 graph พิสูจน์ได้หรือไม่ว่า merge จะเกิด file conflict? เพราะอะไร?

S3.6 `git fetch origin` อาจอัปเดตอะไร และจะไม่เลื่อนอะไรให้โดยอัตโนมัติ?